

Сервис буккроссинга — обмена книгами между пользователями

Данная презентация представляет собой обзор веб-приложения для организации книгообменного пункта. Проект направлен на автоматизацию учёта книг в обороте, выдачи и приёма возвратов через удобный веб-интерфейс.



Идея, цель и архитектура

Основная цель

Автоматизация работы книгообменного пункта — учёт книг в обороте, выдача читателям и приём возвратов через простой веб-интерфейс.

Тип приложения

Single Page Application (SPA) — быстрая навигация между разделами без перезагрузки страницы.

Концепция буккроссинга

Модель книгообменного пункта: администратор вводит книги в оборот, читатели берут и возвращают книги через заявки, которые подтверждает администратор.



Возможности для всех пользователей

Система предоставляет базовые функции, доступные как авторизованным пользователям, так и гостям — обеспечивая открытость и прозрачность книгообменного пункта.



Каталог книг

- Просмотр всех книг в обороте
- Поиск по названию и автору
- Фильтрация по жанру и статусу



Список читателей

- Участники книгообменного пункта
- Просмотр без регистрации



Фильтрация и поиск

- По жанру
- По статусу книги (свободна / на руках)

Функции для читателей и администратора

Система реализует разграниченный доступ: читатели управляют своими заявками, а администратор контролирует весь процесс обмена книгами.

Читатель



- Взять книгу
 - Оформление заявки «Взять себе»
 - Доступно для свободных книг
- Вернуть книгу
 - Оформление заявки на возврат
 - Для книг, которые на руках
- Мои заявки
 - История заявок и их статусы

Администратор

- Управление книгами (CRUD)
 - Добавление, редактирование, деактивация книг
- Управление жанрами и читателями (CRUD)
 - Создание и редактирование жанров
 - Выдача читательских билетов
- Обработка заявок
 - Одобрение/отклонение заявок на выдачу и возврат

Технологический стек Frontend

Frontend-часть приложения реализована с использованием современных технологий, обеспечивающих высокую производительность, надёжность кода и удобство разработки.



React 19.2.0

Библиотека для создания пользовательского интерфейса.

2

TypeScript 5.9.3

Типизированный JavaScript для повышения надёжности кода.



Vite (rolldown-vite 7.3.0)

Современный инструмент сборки для быстрой разработки.



MobX 6.15.0

Библиотека для управления состоянием приложения.

5

SCSS/SASS 1.97.1

Препроцессор CSS для модульной стилизации.



Firebase 12.7.0

Firebase Realtime Database для хранения всех данных приложения.

Структура проекта: Frontend

Проект имеет чётко организованную структуру, разделённую по функциональным слоям, что облегчает разработку, поддержку и масштабирование.

Основные разделы

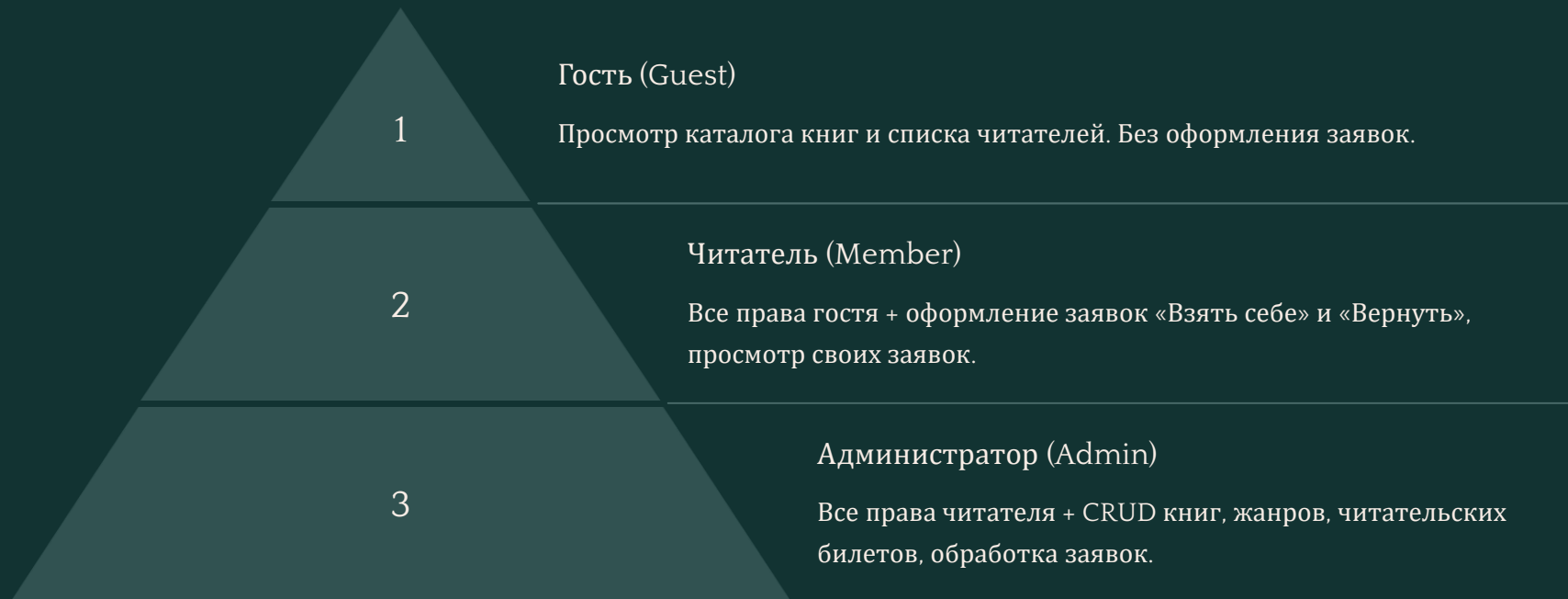
- **components/**: Переиспользуемые UI компоненты (Button, Input, Modal, Table, Badge).
- **pages/**: Страницы приложения (Home, Books, Requests, Members, Admin).
- **store/**: MobX-сторы — AuthStore, DataStore, NavigationStore, UIStore.
- **types/**: TypeScript типы (book, member, request, genre, navigation).
- **styles/**: Глобальные стили, переменные, миксины.
- **firebase.ts**: Конфигурация Firebase и сервис RTDB.
- **App.tsx**: Главный компонент приложения.

Инструменты разработки

- **ESLint** — линтер для проверки качества кода.
- **TypeScript ESLint** — статическая проверка типов.

Система ролей и прав доступа

Проект реализует трёхуровневую систему ролей. Читатели идентифицируются по номеру читательского билета (без паролей), администратор — по паролю.



Основные сущности данных

Данные в системе организованы в четыре ключевые сущности, обеспечивающие полный жизненный цикл книги в обороте.

1

Genre (Жанр)

id, name, description, isActive

2

Member (Читатель)

id, cardNumber, firstName, lastName,
email, phone, joinedDate, isActive

3

Book (Книга)

id, title, author, genreId, condition,
description, status
(available/reserved/taken), holderId,
isActive

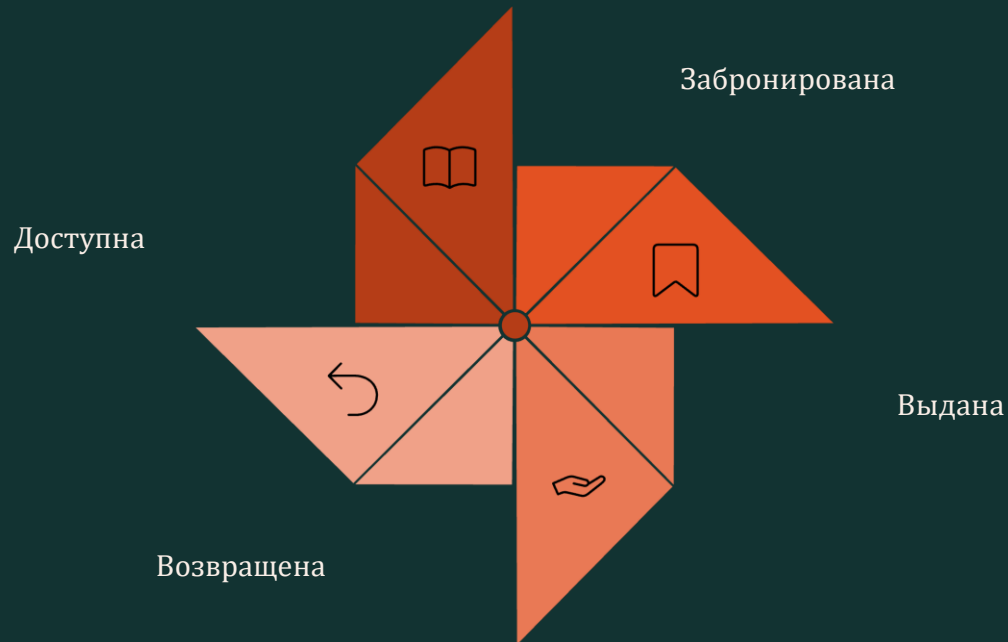
4

BookRequest (Заявка)

id, bookId, memberId, type (take/return), status (pending/approved/rejected), createdAt, resolvedAt

Жизненный цикл книги

Каждая книга проходит строго определённый цикл состояний — от поступления в оборот до повторной доступности для следующего читателя.



Каждое действие читателя оформляется как заявка. Смена статуса книги происходит только после подтверждения администратором, что обеспечивает контроль над оборотом.

Запуск проекта

Для локального запуска проекта необходимо выполнить несколько простых шагов. Требования: Node.js версии 18 или выше, npm.

01

Клонирование репозитория

Загрузите исходный код проекта из репозитория.

02

Установка зависимостей

```
npm install
```

Установите все необходимые пакеты и библиотеки.

03

Настройка Firebase

Сконфигурируйте Firebase Realtime Database в файле `firebase.ts`.

04

Запуск Development сервера

```
npm run dev
```

Запустите сервер для разработки. Учётные данные для входа — в файле `CREDENTIALS.md`.